

第三章--自下而上语法分析

自下而上语法分析方法的基本思想是移进—归约。归约是推导的逆过程。



回顾 LR(0) 项目集规范族



- 1. 拓广文法：在原文法 $G[S]$ 上增加一个产生式 $S' \rightarrow S$ ，这是为了得到唯一的接受状态 $S' \rightarrow S \cdot$ 。
- 2. 设项目集规范族 C 只包含第一个状态 $\{S' \rightarrow \cdot S\}$ 的闭包，即 $C = \{ \text{Closure}(\{S' \rightarrow \cdot S\}) \}$
- 3. 利用 $G0$ 函数对 C 中的每个项目集和每个符号 X 计算其下一状态，并将下一状态 $G0(I, X)$ 加入到 C 中，直到 C 中状态数不再增加
- C 即为文法 G' 的 $LR(0)$ 项目集规范族

回顾 LR(0) 文法分析表的构造



设有文法G，则从LR(0)项目集规范族构造分析表的方法为：

- ① 对于 $A \rightarrow \alpha \cdot X\beta \in I_k$, $G0(k, X) = I_j$
若 $X \in V_t$, 则置 $action[k, x] = S_j$, 即把 (j, a) 移进栈
若 $X \in V_n$, 则置 $goto[k, X] = j$
- ② 对于 $A \rightarrow \alpha \cdot \in I_k$, 则对所有的 $x \in V_t$ 和 $\#$, 均置 $action[k, x] = r_j$ (设 $A \rightarrow \alpha$ 是第 j 个产生式), 即用 $A \rightarrow \alpha$ 归约
- ③ 若 $S' \rightarrow S \cdot \in I_k$, 则置 $action[k, \#] = acc$, 即接受
- ④ 其他均置出错。

回顾 SLR(1) 分析表的构造

设有文法 G ，则SLR(1)分析表的构造方法为：

- ① 对于 $A \rightarrow a \cdot X\beta \in I_k$ ， $G0(I_k, X) = I_j$
若 $X \in V_t$ ，则置 $action[k, X] = S_j$ ，即把 (j, a) 移进栈
若 $X \in V_n$ ，则置 $goto[k, X] = j$
- ② 对于 $A \rightarrow a \cdot \in I_k$ ，则对所有的 x ， $x \in follow(A)$ ，
则置 $action[k, x] = r_j$ （设 $A \rightarrow a$ 是第 j 个产生式），即用 $A \rightarrow a$ 归约
- ③ 若 $S' \rightarrow S \cdot \in I_k$ ，则置 $action[k, \#] = acc$ ，即接受
- ④ 其他均置出错。

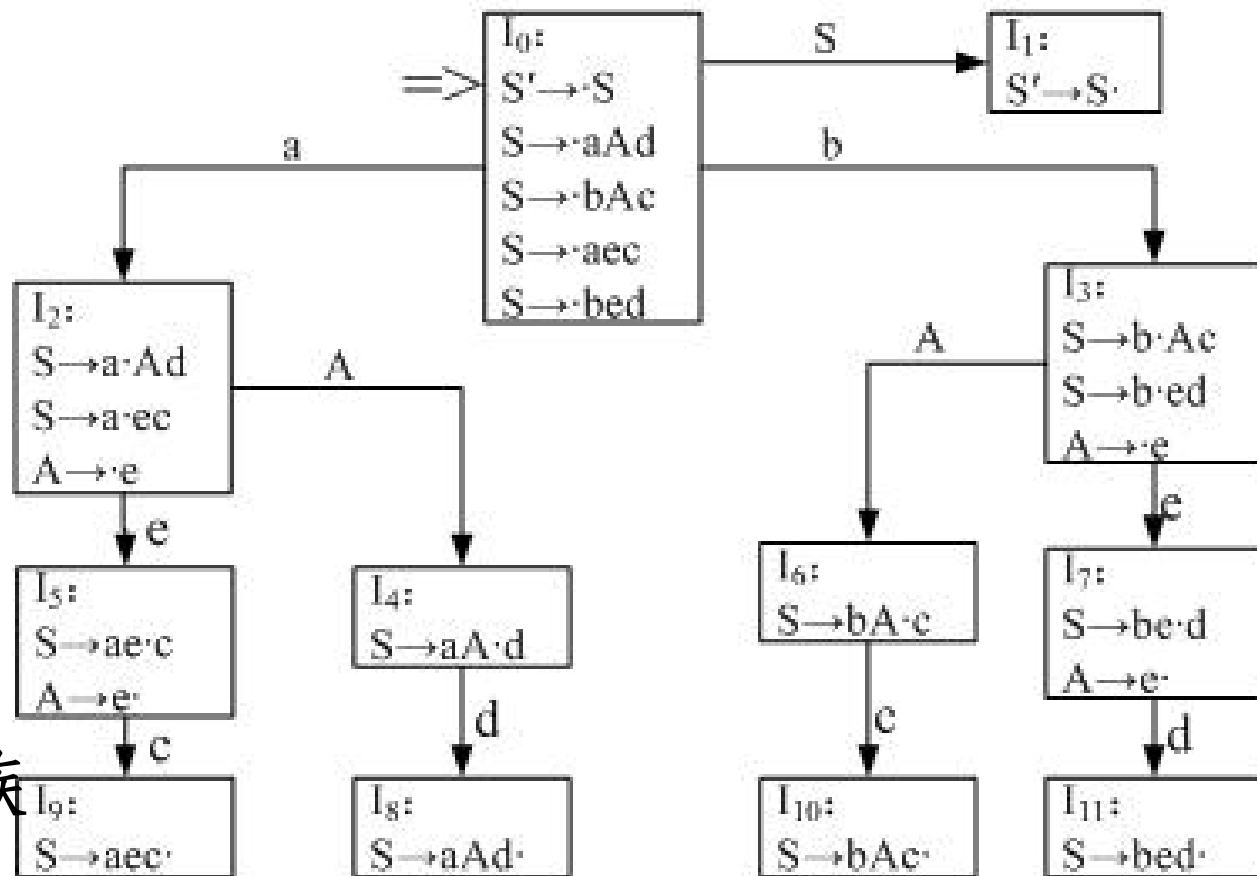
如果分析表的每个入口不含多重定义，则称为SLR表，文法 G 称为SLR(1)文法。

SLR不能解决的问题

• 拓广文法 G' 为：

- (0) $S' \rightarrow S$
- (1) $S \rightarrow aAd$
- (2) $S \rightarrow bAc$
- (3) $S \rightarrow aec$
- (4) $S \rightarrow bed$
- (5) $A \rightarrow e$

得到项目集规范族



项目集 I_5 和 I_7 中的冲突，不能用SLR(1)方法解决。
 $\text{Follow}(A) = \{c, d\}$

4 LR(1)分析

- **LR(k)项目**：为 $[A \rightarrow \alpha \cdot \beta, a_1 a_2 \cdots a_k]$ ， $A \rightarrow \alpha \cdot \beta$ 是一个LR(0)项目， $a_i \in V_T^*$ 。
- $a_1 a_2 \cdots a_k$ 称为它的向前搜索字符串。



- **LR(1)项目**：为 $[A \rightarrow \alpha \cdot \beta, a_1]$ ， $A \rightarrow \alpha \cdot \beta$ 是一个LR(0)项目， $a_1 \in V_T$ 。
- a_1 称为它的向前搜索字符串。

1) LR(1)分析的基本概

念

- 一个LR(1)项目集的闭包 $\text{Closure}(I)$:

(1) 将 I 中的所有项目都加入 $\text{Closure}(I)$ 。

(2) 若项目 $[A \rightarrow \alpha \cdot B \beta, a] \in \text{Closure}(I)$, $B \rightarrow \gamma$ 是一个产生式, 那么对于任何 $b \in \text{First}(\beta a)$, 如果 $[B \rightarrow \cdot \gamma, b]$ 原来不在 $\text{Closure}(I)$ 中, 则把它加进去。

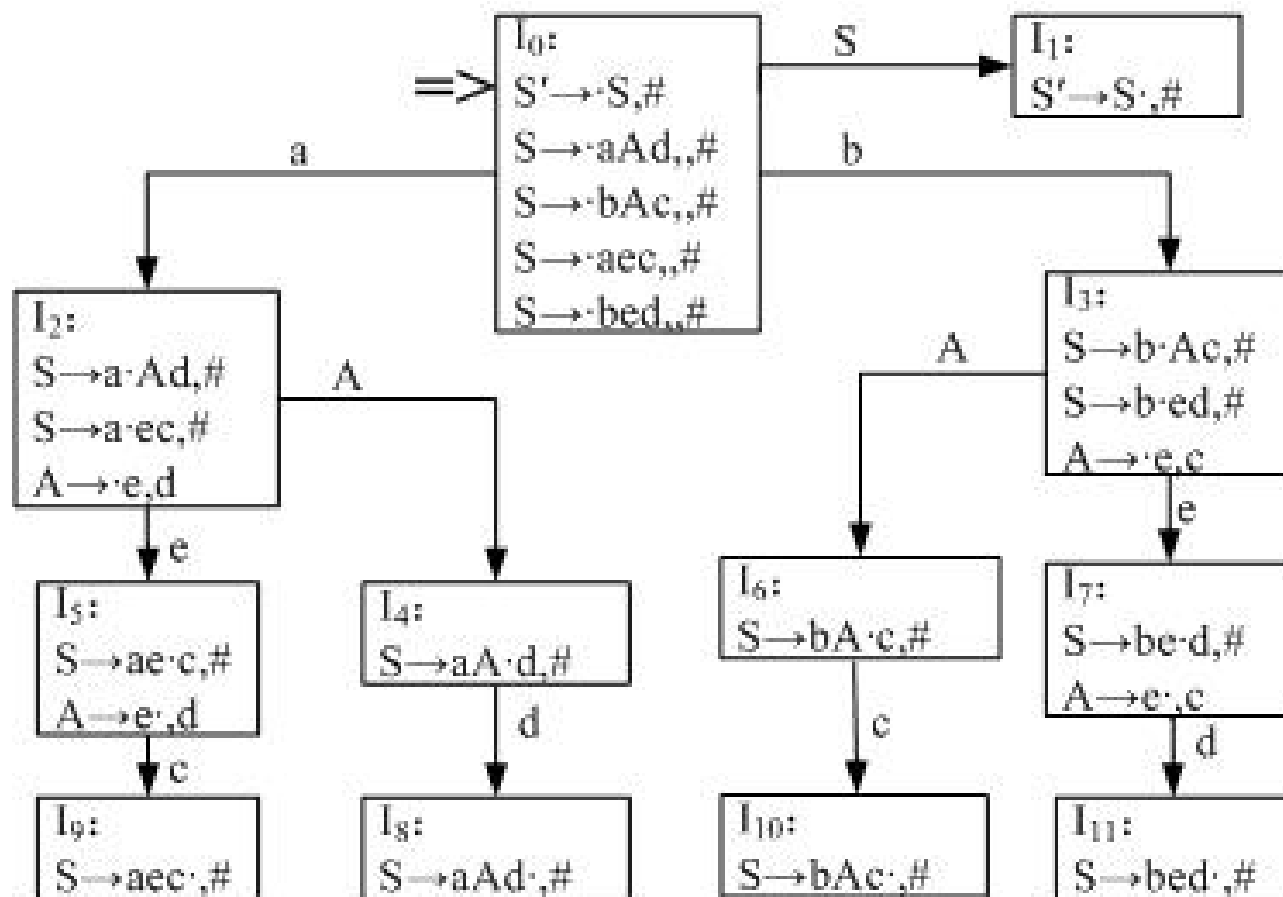
重复执行该过程, 直到 $\text{Closure}(I)$ 不再增大为止。

- I 是一个项目集, X 是一个文法符号, 则转换函数 $\text{GO}(I, X)$ 定义为: $\text{GO}(I, X) = \text{Closure}(J)$, $J = \{ \text{任何形如 } [A \rightarrow \alpha X \cdot \beta, a] \text{ 的项目} \mid [A \rightarrow \alpha \cdot X \beta, a] \in I \}$ 。

2) LR(1)项目集规范族的构造

- (1) $C := \{\text{Closure}(\{[S' \rightarrow \cdot S, \#]\})\};$
- (2) 重复执行(3)的动作, 直到C不再增大;
- (3) FOR C中的每个项目集I和G'的每个符号X DO
 {
 IF $G_0(I, X)$ 非空且不属于C
 把 $G_0(I, X)$ 加入C中;
 在I和 $G_0(I, X)$ 之间画标记为X的弧线;
 }

文法3.8的LR(1)项目集规范族



3) LR(1)分析表的构造

- (1) 若项目 $[A \rightarrow \alpha \cdot a \beta, b] \in I_k$, 且 $G0(I_k, a) = I_j$, 其中 $a \in V_T$, 则置 $action[k, a] = S_j$ 。即把输入符号 a 和状态 j 分别移入文法符号栈和状态栈。
- (2) 若项目 $[A \rightarrow \alpha \cdot, a] \in I_k$, 其中 $a \in V_T$, 则置 $action[k, a] = r_j$, 即用产生式 $A \rightarrow \alpha$ 进行归约, j 是在文法中对产生式 $A \rightarrow \alpha$ 的编号。
- (3) 若项目 $[S' \rightarrow S \cdot, \#] \in I_k$, 则置 $action[k, \#] = "acc"$, 表示"接受"。
- (4) 若 $G0(I_k, A) = I_j$, 其中 $A \in V_N$, 则置 $goto[k, A] = j$ 。表示当栈顶符号为 A 时, 从状态 k 转换到状态 j 。
- (5) 凡不能用规则(1)~(4)填入分析表中的元素, 均置"报错标志"。



文法
3.8
的分析表

状态	action						goto	
	a	b	c	d	e	#	S	A
0	S ₂	S ₃					1	
1						acc		
2					S ₅			4
3					S ₇			6
4				S ₈				
5			S ₉	r ₅				
6			S ₁₀					
7			r ₅	S ₁₁				
8						r ₁		
9						r ₃		
10						r ₂		
11						r ₄		

5 LALR(1) 分析

- LALR分析方法是一种折衷方法，它的分析表比LR(1)分析表要小得多，能力也弱一些，但它能应用在一些SLR(1)不能应用的场合。实际的编译器经常使用这种方法，大多数程序设计语言的语法结构能方便地由LALR文法表示。



- 在LR(1)分析表中，若存在两个状态（项目集）除向前搜索符不同外，其它部分都是相同的，称这样的两个LR(1)项目集是同心的。



- 如果把同心的LR(1)项目集合并，心仍相同（心就是一个LR(0)项目集），超前搜索符集为各同心集超前搜索符的并集，合并同心集后go函数自动合并。称为LALR方法。

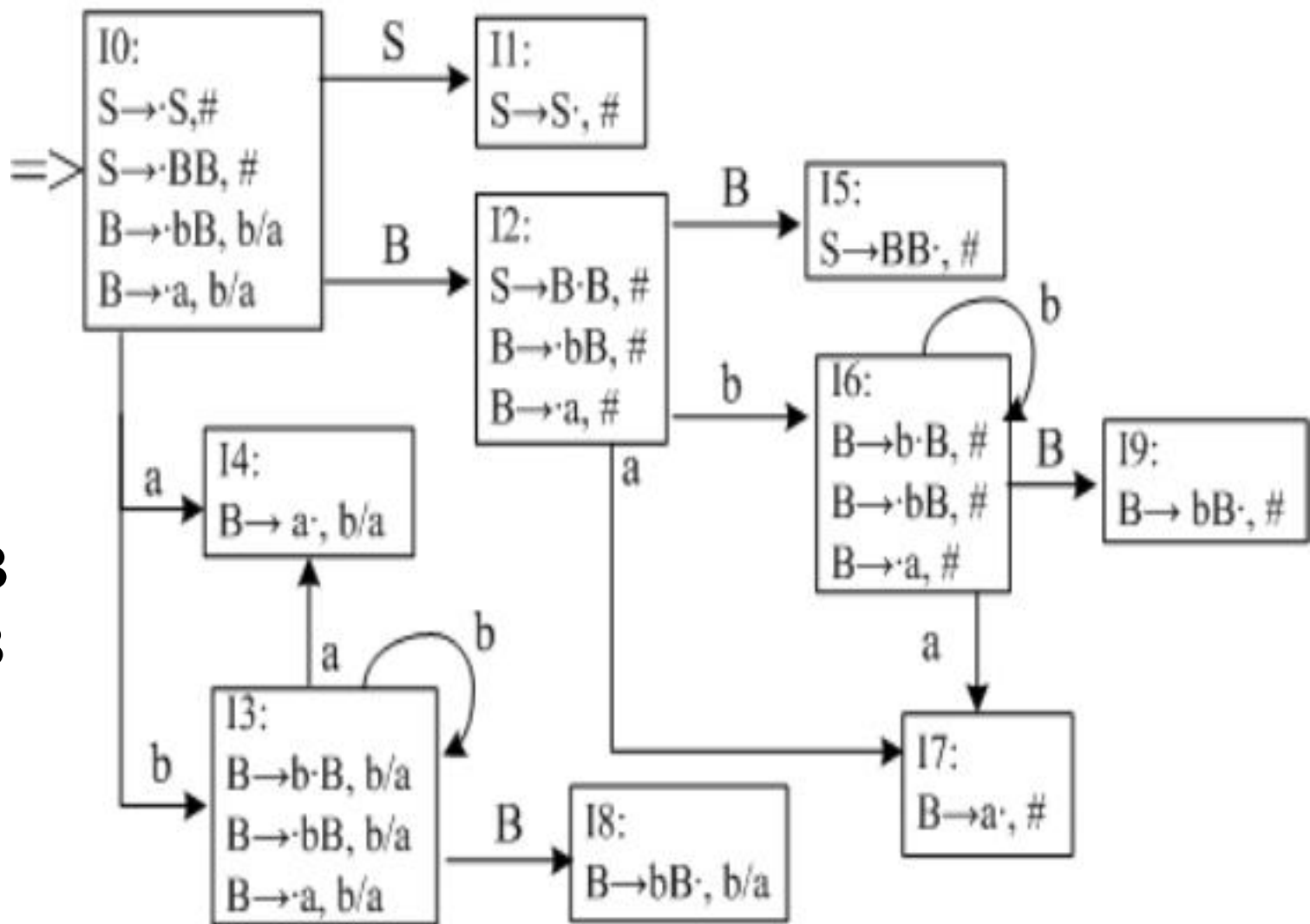
构造LALR分析表的算法：

- (1) 构造LR(1)项目集规范族, $C=\{I_0, I_1, \dots, I_n\}$ 。
- (2) 合并所有的同心集, 得到LALR(1)的项目集族 $C'=\{J_0, J_1, \dots, J_m\}$ 。含有项目 $[S' \rightarrow \cdot S, \#]$ 为初态。
- (3) 由 C' 构造动作(action)表。
 - ① 若 $[A \rightarrow \alpha \cdot a \beta, b] \in J_k$, 且 $G0(J_k, a) = J_j$, 其中 $a \in V_T$, 则置 $action[k, a]=S_j$,
 - ② 若项目 $[A \rightarrow \alpha \cdot, a] \in J_k$, 其中 $a \in V_T$, 则置 $action[k, a]=r_j$, r_j 的含义是按产生式 $A \rightarrow \alpha$ 进行归约
 - ③ 若项目 $[S' \rightarrow S \cdot, \#] \in I_k$, 则置 $action[k, \#]=acc$, 表示分析成功, 接受。

构造LALR分析表的算法：

(4) goto表的构造。若不是同心集的项目集，转换函数的构造与LR(1)的相同；假定 $I_{i1}, I_{i2}, \dots, I_{in}$ 是同心集，合并后的新集为 J_k ，转换函数 $G0(I_{i1}, X), G0(I_{i2}, X), \dots, G0(I_{in}, X)$ 也为同心集，将其合并后记作 J_i ，因此，有 $G0(J_k, X) = J_i$ ，所以当 X 为非终结符时， $G0(J_k, X) = J_i$ ，则置 $goto[k, X] = i$ ，表示在 k 状态下遇到非终结符 X 时，把 X 和 i 分别移到文法符号栈和状态栈。

(5) 分析表中凡不能用(3)、(4)填入信息的空白均填上“出错标志”。



该文法的LR(1)项目集规范族



LR(1)分析表为:

I_3 和 I_6 , I_4 和 I_7 , I_8 和 I_9 分别为同心集, 将同心集合并后为:

$I_{36}: B \rightarrow a \cdot B, a/b/\#$

$B \rightarrow \cdot aB, a/b/\#$

$B \rightarrow \cdot b, a/b/\#$

$I_{47}: B \rightarrow b \cdot, a/b/\#$

$I_{89}: B \rightarrow aB \cdot, a/b/\#$

同心集合并后仍不包含冲突,
因此该文法是LALR文法。

状态	action			goto	
	b	a	#	S	B
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

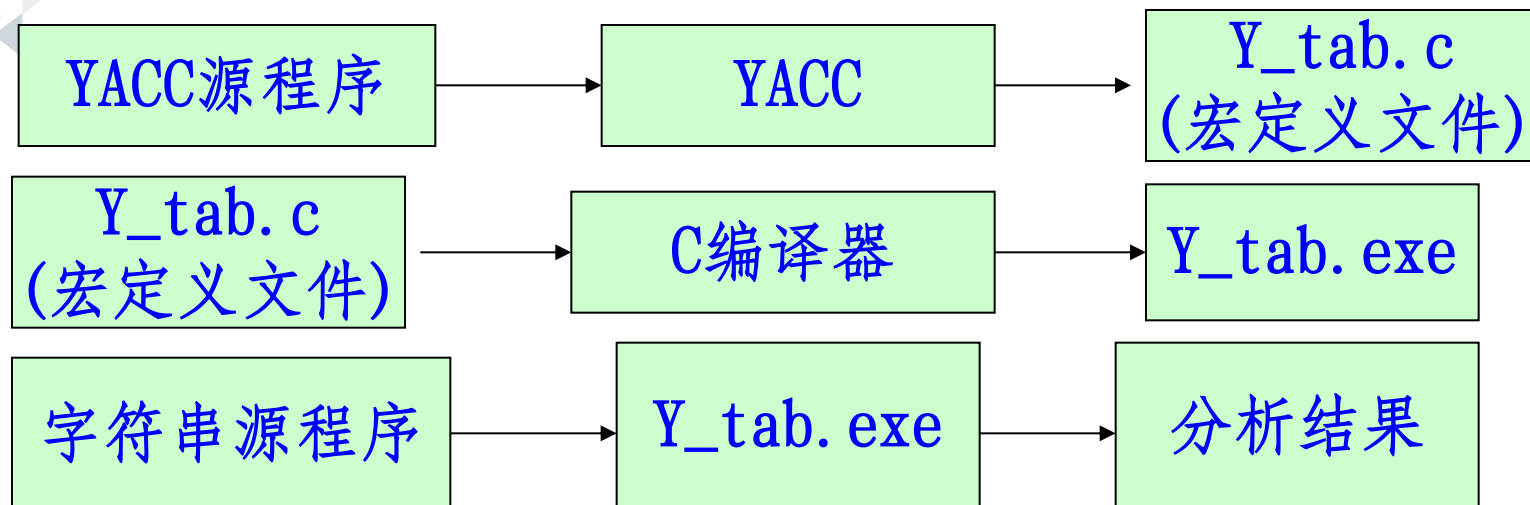
得到LALR分析表：

状态	action			goto	
	b	a	#	S	B
0	$S_{3,6}$	$S_{4,7}$		1	2
1			acc		
2	$S_{3,6}$	$S_{4,7}$			5
3,6	$S_{3,6}$	$S_{4,7}$			8,9
4,7	r_3	r_3	r_3		
5			r_1		
8,9	r_2	r_2	r_2		

3.5 语法分析器的自动生成

- YACC (Yet Another Compiler-Compiler) 是美国贝尔实验室开发的语法分析程序自动生成器
- 输入是某个语言的语法规则，输出是该语言的语法分析器。 目前YACC生成的是一个LALR(1)分析器
- 常用版本：Berkeley大学的BYACC，GNU工程的BISON

语法分析器的自动生成



YACC的输入文件称为YACC源文件，以.y为扩展名，输出文件有两个，一个是包含有语法分析函数yyparse()的C程序，另一个是包含源文件中所有终结符编码的宏定义文件，扩展名为.h。这两个文件经过C编译器的编译就生成一个语法分析器，该语法分析器的输入是源程序经过词法分析的结果，输出可以是一棵语法树，或者是所生成的目标代码，也可以是关于输入串是否符合语法规则的信息。具体的输出形式可以在YACC源程序中自己定义。

语法分析器的自动生成

- YACC和LEX有直接的接口，由于YACC与LEX的特殊功能，这两个姐妹程序成为软件工程的重要工具，被称为“黄金组合”。
- 很多程序设计语言编译程序的设计都使用了LEX和YACC，比如著名的GNU C语言编译器，PASCAL语言向C语言的转换工具p2c等，就是用FLEX和BISON实现的
- 多数程序设计语言的语法分析都采用LALR(1)分析法，YACC也正是以LALR(1)文法为基础。它通过对输入的形式语法规则进行分析，产生LALR(1)分析表，输出以该分析表驱动的语法分析器C语言源程序。



YACC源程序结构

YACC源程序由三个部分组成，各部分以“%%”为分隔符。说明部分和程序部分可选，规则部分是必需的。

[说明部分]

%%

规则部分

[%%

程序部分]



YACC源程序结构—说明部分

- YACC源程序**说明部分**定义语法规则中要用的终结符号，语义动作中使用的数据类型、变量、语义值的联合类型以及语法规则中运算符的优先级等。说明部分可以是空的。

- 说明部分通常包含两部分内容：

C语言代码部分

Yacc说明部分



YACC源程序结构—说明部分

%{

头文件表

宏定义

数据类型定义

全局变量定义

%}

文法开始符号定义

语义值类型定义

终结符定义

非终结符定义

优先级和结合性定义



1-头文件表

- yacc直接把这部分定义抄到所生成的C语言程序y.tab.c去的，所以要按C语言的语法规定来写。头文件表是一系列C语言的#include语句，要从每行的第一列开始写，例如：

```
%{  
#include <stdio.h>  
#include <math.h>  
#include <ctype.h>  
#include "header.h"  
%}  
...  
%}
```

 2-宏定义

- 这部分用C语言的 `#define`语句定义程序中要用的宏
- 例如

```
%{
```

```
...
```

```
#define max(x,y)((x>y)?x:y)
```

```
...
```

```
%}
```



3-数据类型定义

- 这部分定义语义动作中或程序段部分中要用到的数据类型
- 例如

%{

...

```
typedef struct interval{
```

```
double lo,hi;
```

```
}INTERVAL;
```

...

%}



4-全局变量定义

- 外部变量（external variable）和yacc源程序中要用到的全局变量都在这部分定义

- 例如

%{

...

extern int nfg;

douhle dreg[26];

INTERVAL Vreg[26];

...

%}



5-语法开始符定义

- 上下文无关文法的开始符号是一个特殊的非终结符，所有的推导都从这个非终结符开始
- 在yacc中，语法开始符定义语句是
`% start 非终结符.....`
- 如果没有上面的说明，yacc自动将语法规则部分中第一条语法规则左部的非终结符作为语法开始符



6-语义值类型定义

- yacc生成的语法分析程序yparse用的是LR分析方法，它在作语法分析时除了有一个状态栈外，还有一个语义值栈
- 语义值栈存放它所分析到的非终结符和终结符的语义值，这些语义值有的是从词法分析程序传回的，有的是在语义动作中赋予的
- 如果没有对语义值的类型做定义，那么yacc认为它是整型（int）的，即所有语法符号如果赋予了语义值，则必须是整型的，否则会出类型错



6-语义值类型定义

- 但是用户经常会希望语义值的类型比较复杂，如双精度浮点数，字符串或树结点的指针
- 这时就可以用语义值类型定义进行说明。因为不同的语法符号的语义值类型可能不同，所以语义值类型说明就是将语义值的类型定义为一个联合（Union），这个联合包括所有可能用到的类型（各自对应一个成员名）
- 为了使用户不必在存取语义值时每次都指出成员名，在语义值类型定义部分还要求用户说明每一个语法符号（终结符和非终结符）的语义值是哪一个联合成员类型



6-语义值类型定义

- 例:

```
% union{  
int ival  
double dval  
INTERVAL vval;  
}
```

- 引用时候的方式

```
%token <ival> DREG VREG  
%token <dval> CONST  
%type <dval> dexp  
%type <vval> vexp
```

以**%token**开始的行定义的是终结符的类型

以**%type**开始的行定义是非终结符的类型



7-终结符定义

- 在yacc源程序语法规则部分出现的所有终结符（正文字符“+”，“-”等除外）等必须用`%token`定义，定义形式：

- 单一数据类型：

`%token 终结符1 终结符2`

多数据类型：

`%token <类型> 终结符1 终结符2 ...`



8-终结符定义

⊕ 优先级和结合性定义

%left 左结合

%right 右结合

%nonassoc 无结合性

%prec <终结符> 强制定义优先级



YACC源程序—语法规则部分

- 语法规则部分是整个YACC源程序的主体，它是由一组产生式及相应的语义动作组成。
- 规则部分包括修改的BNF格式的文法规则，以及将在识别出识别出相关的文法规则时被执行的C代码中的动作（即根据LALR（1）分析算法，在归约中使用）。
- 文法规则中使用的元符号惯例如下：

通常，竖线|被用作替换（也可以分别写出替换项），而用来分隔文法规则的左右两边的箭头符号->在YACC中用冒号表示，最后，必须用分号来结束每个文法规则。



YACC源程序—语法规则部分

对文法中的产生式 $A \rightarrow \alpha_1 | \alpha_2 | \cdots | \alpha_m$

在YACC程序中可表示成

A: α_1 {语义动作1}
 $| \alpha_2$ {语义动作2}

 $| \alpha_m$ {语义动作m}
;



YACC源程序—语法规则部分

•例：文法规则和动作：

$\text{exp}:\text{exp}' + \text{term}\{\$\$=\$1+\$3;\}$

含义是：当识别规则 $\text{exp} \rightarrow \text{exp} + \text{term}$ 时，左边 exp 值为右边的 exp 的值与右边的 term 的值之和，其中 $\$\$$ 代表规则左部符号 exp 的值， $\$1$ 代表规则右部第一个符号 exp 的值、 $\$3$ 表示规则右部第三个符号 term 的值。



YACC源程序—程序部分组成

- YACC源程序的程序部分包括：
 - 主程序 `main()`
 - 错误信息执行程序 `yyerror(s)`
 - 词法分析程序 `yylex()`, 可以与LEX进行整合
 - 用户在语义动作中用到的子程序
- YACC约定：
 - 传递词法分析程序token属性值的全程变量名: `yyval`
 - 生成的语法分析程序名为: `yyparse()`;



二义性文法的处理

- YACC生成LALR(1)分析器，如果接受的文法不是LALR(1)分析表就有冲突。YACC解决冲突的默认规则为：
 - 归约—归约冲突：选择YACC源程序中排列在前面的产生式进行归约；
 - 移进—归约冲突：移进动作优先于归约动作。

3.6 语法分析中的错误处理

- 语法出错处理概论
- 常见错误的分析处理
- 不可预料错误的处理



3.6.1 语法分析中出错处理原则

- 出错处理要解决的主要问题
 - 出错 (procedure error (n:integer))
指出错误类型、halt
 - 一次运行，尽可能多的发现语法错误



3.6.1 语法分析中出错处理原则

- 出错处理遵循的原则
 - 发现错误为主，校正错误为辅
 - 错误局部化，选择一个适当的位置恢复分析过程
 - 准确报告，尽早给出错误的位置



3.6.1 语法分析中出错处理原则

分析程序可能的出错情况和解决方案：
常见、易犯的错误—预先假设，在程序相应处作修改或扩充



不常见、无法预料的错误—跳过一部分原文，让它在适当的地方停下来，然后继续分析下去

3.6.1 语法分析中出错处理原则

- 第一类错误处理和具体的语言有关
- 第二类错误处理的一些原则：
 - 假如在识别出一个错误之后，要跳过原文的某一部分，那么语言必不可少地要包含一些关键字，这些关键字用错的几率非常小
 - 在出错情况下当时的语法分析过程并不简单地放弃它的目标，而是跳过一部分原文，使被调用的语法分析过程总是能够正常结束

3.6.2 常见错误的分析处理

- 修改语法分析程序
- 修改语法定义
- 增加测试函数和参数



Example 1 :

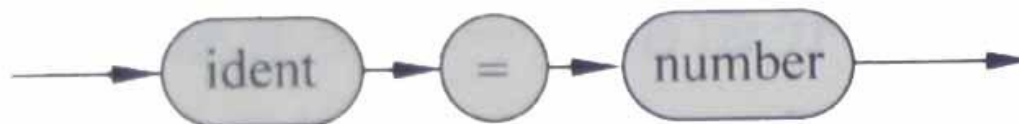
正确: `const m=7,n=85;`

错误: `const m:=7,n=85;`

- 将说明中的等号写成赋值号

- 解决方法: 修改constdeclaration语法分析程序, 使原来接收“=”的, 也可以接收“:=”

constdeclaration语法结构图



```
begin if sym=ident then
  begin getsym;
    if sym in [eq, becomes] then
      begin if sym = becomes then error(1);
        getsym;
        if sym=number then
          begin enter(constant); getsym
        end
      else error(2)
      end else error(3)
    end else error(4)
```


Example2:

正确:

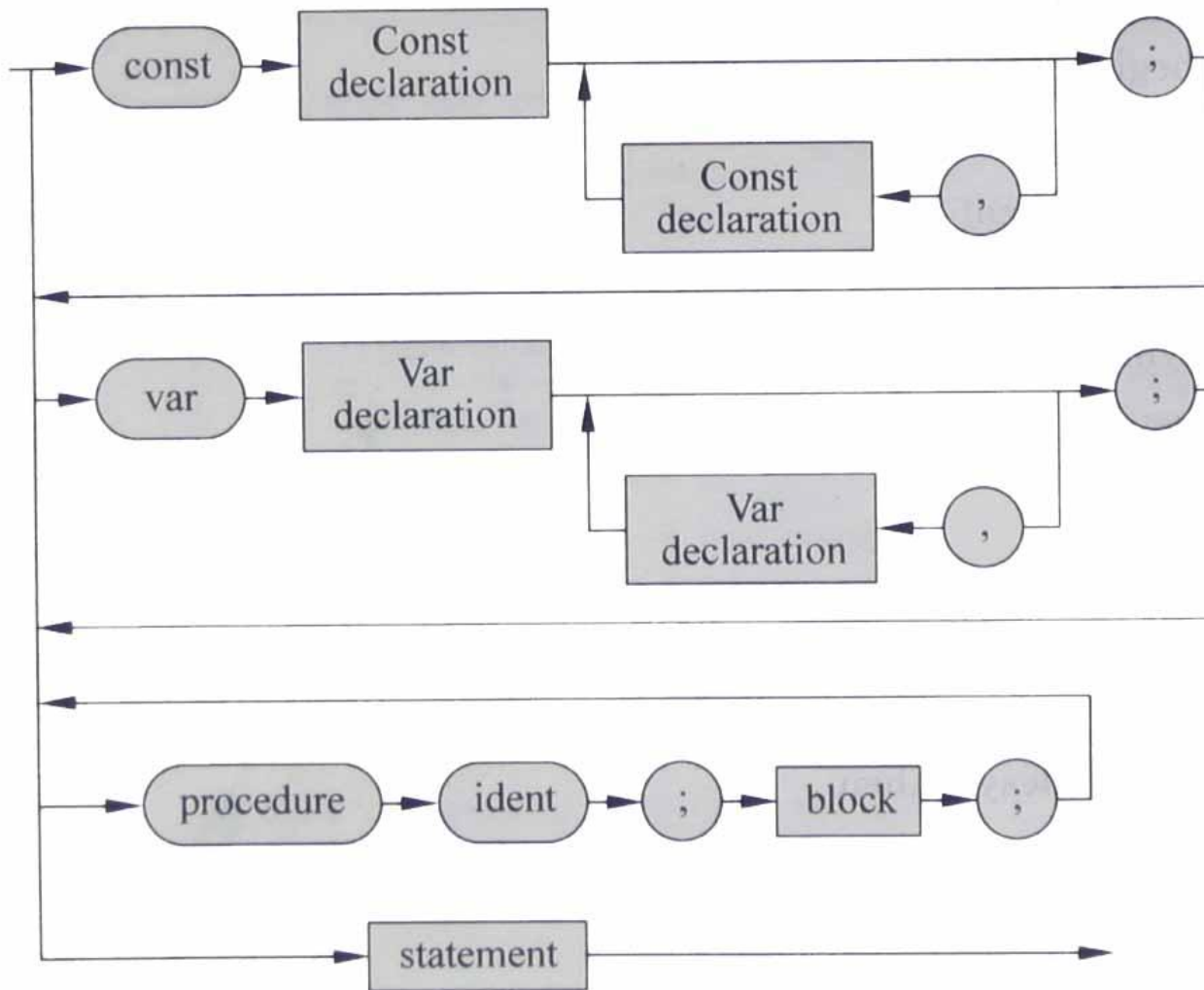
```
const m=7,n=85;
```

```
var x,y,z,q,r;
```

错误:

- const m=7;n=85,
 - const m=7,n=85,
 - const m=7,n=85
-
- 容易犯的错误: 将中间的逗号写成分号; 将结束的分号写成逗号; 遗漏结束时候的分号
 - 解决方法: 修改block语法分析程序

• block语法结构图





repeat

if sym=constsym then

begin getsym;

repeat constdeclaration;

while sym=comma do

begin getsym; constdeclaration

end;

if sym=semicolon then getsym else error(5)

until sym <> ident

end;

if sym=varsym then

begin getsym;

repeat vardeclaration;

while sym=comma do

begin getsym; vardeclaration

end;

if sym=semicolon then getsym else error(5)

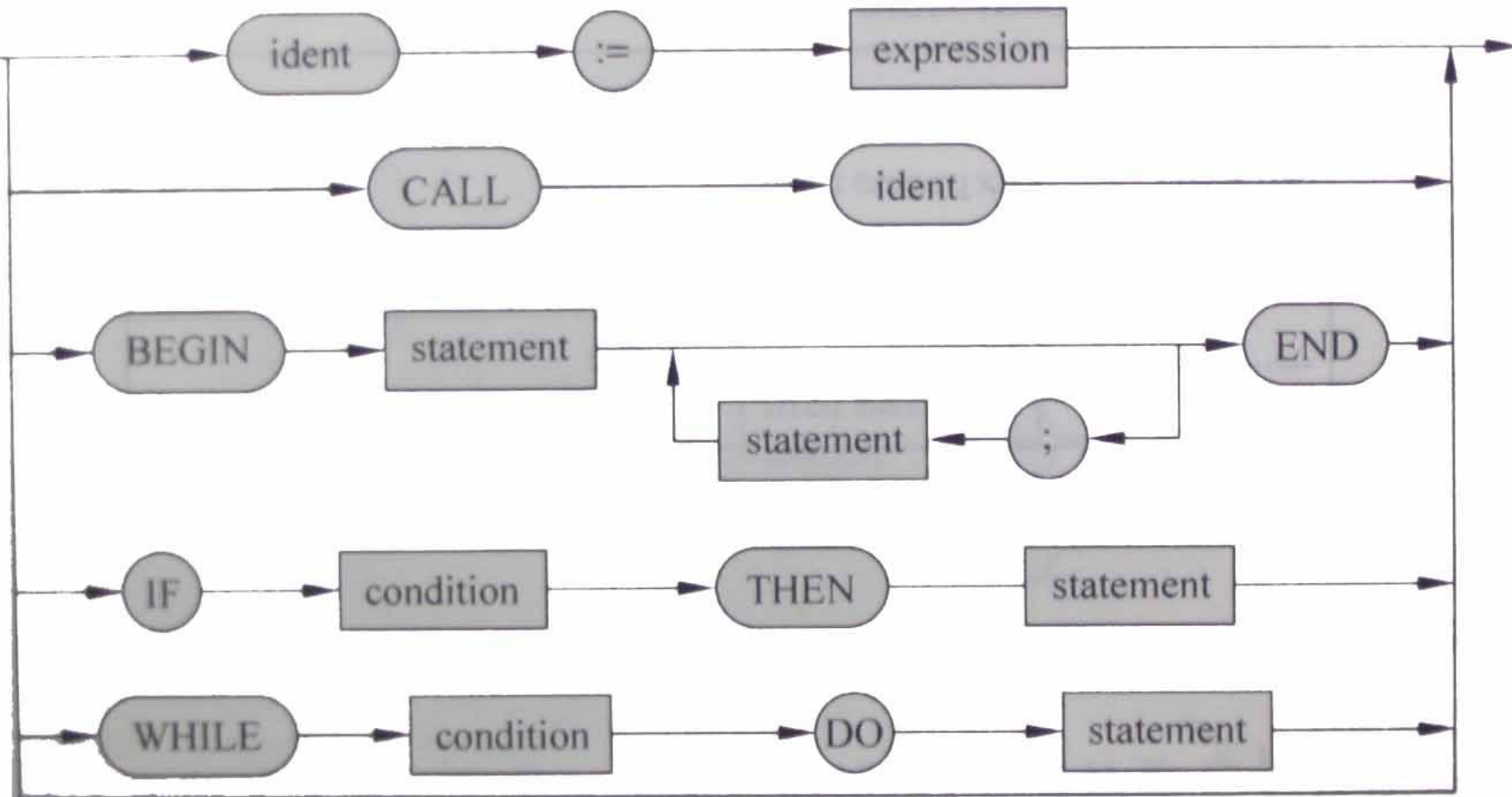


Example 3:

```
begin
    if odd b then
        z:=z+a;
    a:=2*a;
    b:=b/2
end;
```

- 常见错误:语句之间丢失分号
- 解决办法: 修改statement语法分析程序

statement语法结构



```
if sym=beginsym then
```

```
    begin getsym;
```

```
    statement([semicolon,endsym]+fsys);
```

```
    while sym in [semicolon]+statbegsys do
```

```
        begin
```

```
            if sym = semicolon then getsym else  
error(10);
```

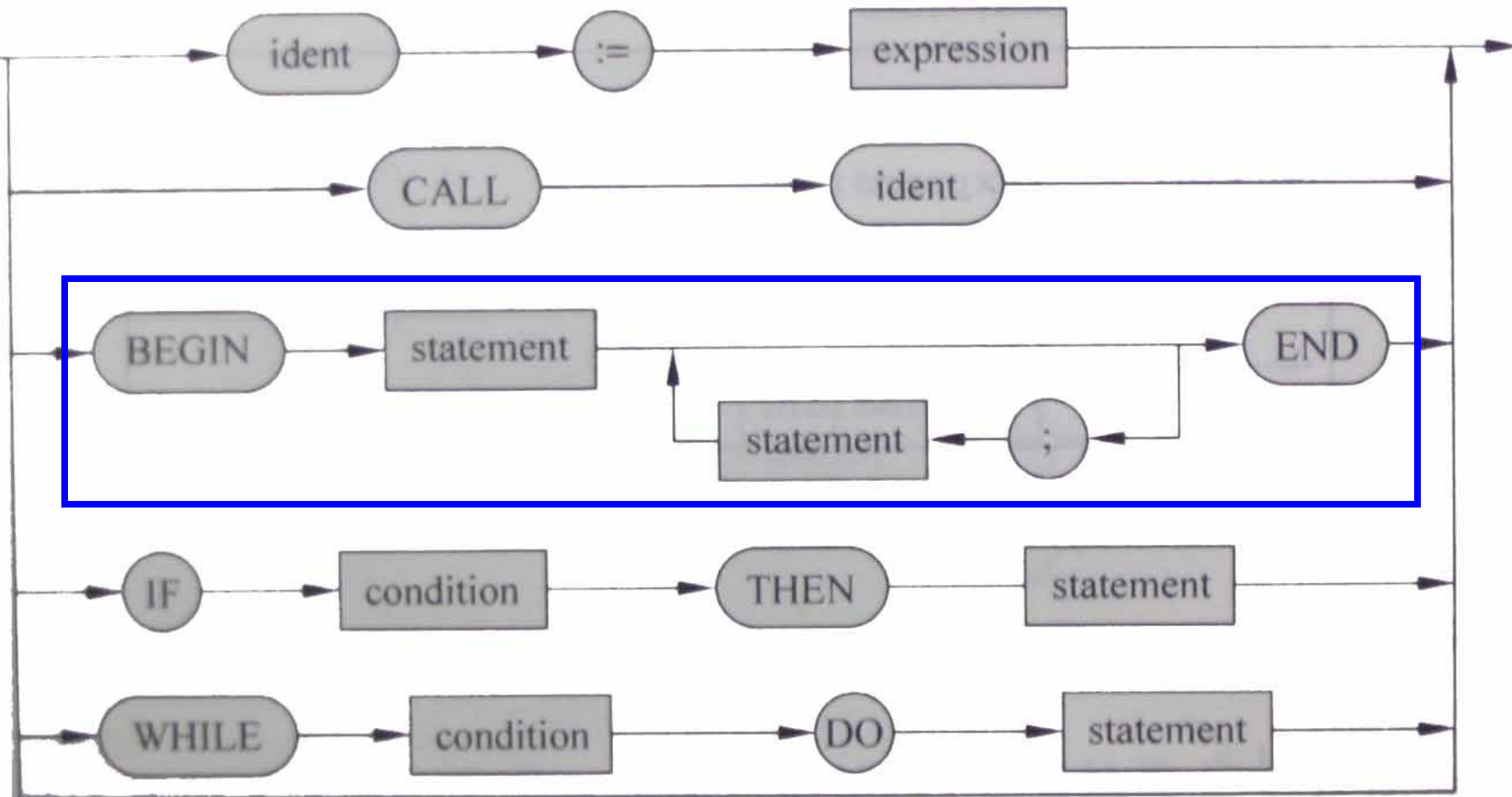
```
            statement([semicolon,endsym]+fsys)
```

```
        end;
```

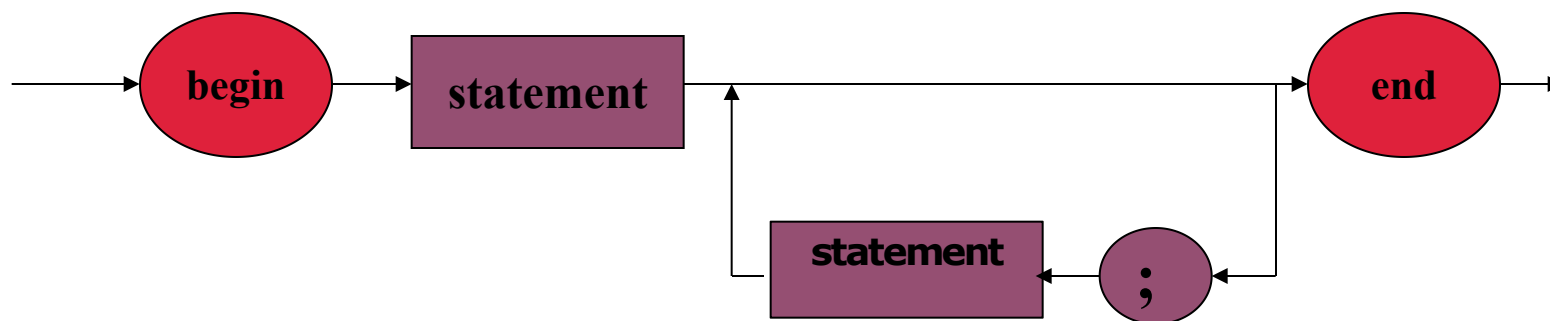
```
        if sym=endsym then getsym else error(17)  
    end else...
```

2

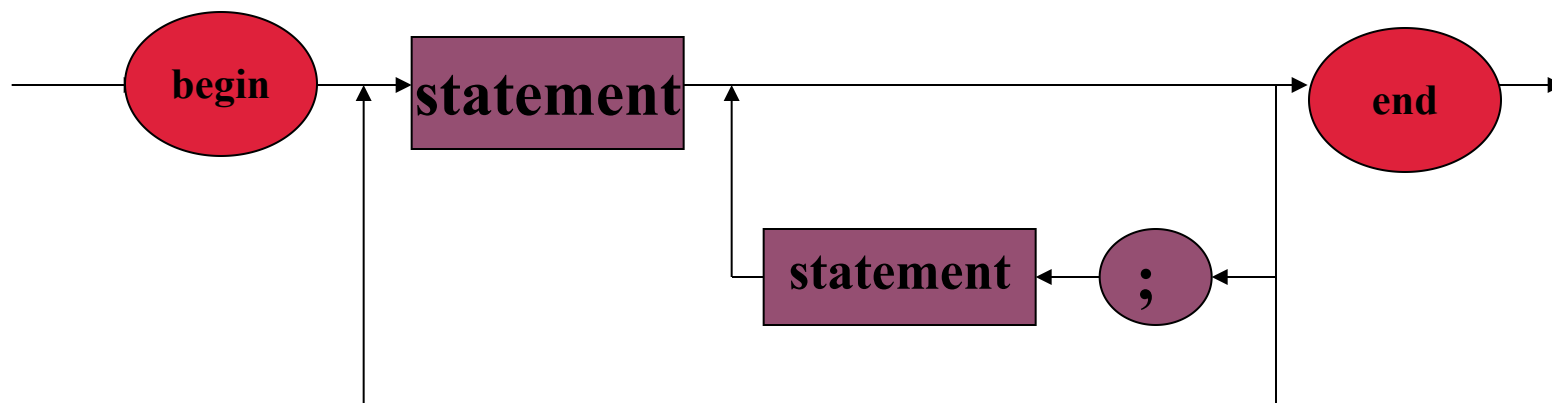
修改语法定义



statement改造前的语法结构图



statement改造后的语法结构图





```
if sym=beginsym then
begin getsym;
  repeat statement([semicolon,endsym]+fsys);
  while sym=semicolon do
    begin getsym;
    statement([semicolon,endsym]+fsys);
  end;
until not (sym in statbegsys)
if sym=endsym then getsym else error()
end;
```

3. 不可预料错误的处理

- 策略：跳过一部分原程序，在适当的地方停下来，然后再继续分析下去
- 在语法定义结构上
 - 尽量简明的语言结构
 - 尽量使每个语法结构的头一个符号都使用关键字
- 在出错处理技术上
 - 每个语法单位的分析过程补充一个参数：
fsys: 表示该语法单位的跟随符集合
 - 增加一个test过程，三个参数s1, s2和n

当语法分析进入某些关键字或终结符号集合为开始符号的语法单元时，在其入口或出口调用一个测试程序TEST。例如，语句的开始符是begin, if, while, cal, read, write; 说明的开始符为var, const, procedure; 因子的开始符是“(”, ident, number。当语法分析进入这样的语法单元前，可用测试程序检查当前单词符号是否属于他们开始符号的集合，若不是则出错。

- test函数 procedure test(s1,s2:symset; n:integer);
begin if not(sym in s1) then
begin error(n); s1:=s1+s2;
while not(sym in s1) do
getsym
end
end(*test*);

TEST测试过程三个参数

- S1：当语法分析进入或退出某一语法单元时当前单词符号应属于的集合，他可能是一个语法单元的开始符号集合，也可能是一个语法单元的后继符号集合。
- S2：在某一出错状态时，可恢复语法分析继续工作的补充单词符号集合。因为当语法分析出错时，即当前单词符号不再集合S1中，为了继续编译，需跳过后遍输入的一写单词符号，直到当前输入的单词符号属于S1和S2其和。
- n：出错信息编号。

PL/0文法非终结符的开始符号与后继符号集合表



非终结符	开始符号集合	后继符号集合
分程序 block	const var Procedure ident if call begin while	. ;
语句 statement	ident call begin if while	. ; end
条件 condition	odd + - (ident number	then do
表达式 expression	+ - (ident number	. ; rop end then do
项 term	ident number (	. ;) rop + - end then do
因子 factor	ident number (	. ;) rop + - * / end then do

- 例如，考察因子的语法分析。在过程FACTOR的入口处调用一次TEST过程，它的实参S1是因子开始符号集合。S2是每个过程的形参FSYS调用时实参的传递值。当编译程序第一次调用BLOCK时，FSYS的实参为[.]与说明开始符号和语句开始符号集合的和。以后随着调用语法分析程序层次的深入逐步增加。如在调用语句时增加了[;]和[endsym]，在表达式语法分析中调用项时又增加[+]和[-]，而在项中调用因子时又增加了[*]和[/]，这样在进入因子分析程序时即使当前符号不是因子开始符，出错后只要跳过一定的符号，遇到当时输入的单词符号在FSYS中或在因子开始符号集合中，均可继续正常进行语法分析。在因子过程的出口处也调用了测试过程，当时的FSYS集合的单词符号都是因子正常出口时允许的单词符号。



小结

- LR(1)分析过程
- LALR(1)分析过程
- 语法分析器自动生成工具
- 语法分析器中的错误处理